UNIVERSIDAD COMPLUTENSE MADRID

Tema 2. Resolución de problemas con búsqueda

2.2. Resolución de problemas mediante búsqueda

Javier Bravo Agapito

Basado en las diapositivas de Virginia Francisco, Rubén Fuentes, Alberto Díaz, Carmen Fernández y Belén Díaz

Curso 2024-25

Contenidos

- Espacio de búsqueda
- Esquema general de búsqueda
- Evaluación de estrategias de búsqueda
- Tipos de estrategias de búsqueda
- Estrategias de búsqueda no informada
  - Búsqueda primero en anchura
  - Búsqueda de coste uniforme
  - Búsqueda primero en profundidad
  - Búsqueda de profundidad limitada
  - Búsqueda primero en profundidad con profundidad iterativa
  - Búsqueda bidireccional

Espacio de búsqueda I

- El espacio de búsqueda (o árbol de búsqueda) solo contiene los nodos generados en la búsqueda de la solución.
- Es generado por el espacio inicial y la aplicación de los operadores.
- Si el mismo estado puede ser alcanzado desde varios caminos -> grafo de búsqueda.
- Búsqueda en espacios de estados:
  - No supone que el grafo esté previamente generado.
  - Ni asume tampoco que se tengan que generar todos los estados.
  - El espacio de estados del 8-puzzle tiene 9!/2 = 181.440 estados...
  - Sólo se generan los estados necesarios según la estrategia de búsqueda.

Espacio de búsqueda II

- Nodo de búsqueda
  - Estado inicial.
  - Raíz del árbol de búsqueda.
- Expandir
  - Resultado de aplicar un operador al nodo actual.
- Generar
  - Al expandir se generan nuevos estados.
- Estrategia de búsqueda
  - Determina el estado a expandir.
- Frontera (nodos abiertos): nodos que se han generado y no expandido.
- Nodo hoja: nodo sin sucesores en el árbol.

Espacio de búsqueda III

- Aunque el espacio de estados sea finito,…el espacio de búsqueda puede ser infinito por los posibles ciclos del grafo
  - Nodos distintos del árbol de búsqueda pueden tener el mismo estado del espacio de estados.
- Estructura de un nodo:
  - estado
  - nodoPadre
  - operador
  - profundidad
  - costeAcumulado
- Los nodos son diferentes de los estados.

Espacio de búsqueda. Control de repeticiones

- Evitar la repetición de estados tiene un coste.
- Puede hacerse a distintos niveles
  1. Evitar aplicación sucesiva de operadores inversos (si hay) -> bajo coste.
  2. Evitar ciclos en el camino actual (guardando estados del camino actual).
  3. Evitar la repetición de un estado en cualquier camino -> se necesita mucho espacio y tiempo para hacer las comparaciones
     - Se marcan los estados que han sido generados para comprobar la no repetición.
- Debe de haber un compromiso entre lo que se intenta evitar y el coste de evitarlo.

Esquema general de búsqueda

- Inicializamos el conjunto de nodos abiertos con el nodo que contiene el estado inicial.
- Repetimos
  - Si conjunto de nodos abiertos está vacío → Devolvemos fallo
  - Extraemos un nodo del conjunto de nodos abiertos → Nodo actual
    - La elección del nodo a extraer dependerá de la estrategia
  - Si el nodo actual contiene el estado objetivo
    - Devolvemos la solución
  - Si el nodo actual no contiene el estado objetivo
    - Expandimos el nodo actual, añadiendo los hijos del nodo actual al conjunto de nodos abiertos

Esquema general de búsqueda con control de ciclos

- Inicializamos el conjunto de nodos abiertos con el nodo que contiene el estado inicial
- Inicializamos el conjunto de nodos cerrados a vacío
- Repetimos
  - Si conjunto de nodos abiertos está vacío → Devolvemos fallo
  - Extraemos un nodo del conjunto de nodos abiertos → Nodo actual
    - La elección del nodo a extraer dependerá de la estrategia
  - Si el nodo actual contiene el estado objetivo
    - Devolvemos la solución
  - Si el nodo actual no contiene el estado objetivo
    - Añadimos nodo actual a cerrados
    - Expandimos el nodo actual, añadiendo los hijos del nodo actual al conjunto de nodos abiertos sólo si no están ni en abiertos ni en cerrados

Esquema general de búsqueda. Elementos

- **Nodos Abiertos** (o frontera): nodos pendientes de expandir.
- **Nodos Cerrados**: nodos ya expandidos o visitados (sólo en algunos algoritmos).
- Tipos de estados:
  - **Generados**: aquellos que aparecen o han aparecido en nodos de abiertos.
  - **Generados pero no expandidos**: aquellos que aparecen en la lista de nodos de abiertos.
  - **Expandidos**: un estado se expande cuando
    1. Un nodo que lo representa se quita de abiertos
    2. Se generan todos sus descendientes y éstos se añaden a abiertos
  - Un estado puede ser expandido varias veces (en diferentes nodos).
  - Un nodo se expande una única vez.

Algoritmo general de búsqueda

Función Búsqueda-arboles (problema, frontera) devuelve una solución o fallo{

frontera ← Inserta(Hacer-nodo(estado-inicial[problema]), frontera)

hacer bucle

si Vacia(frontera) devolver fallo

si no,

nodo ← Extraer-nodo(frontera)

si Estado[nodo] está en objetivo devolver Solución(nodo)

si no,

lista-nodos ← Expandir(nodo, problema)

frontera ← Insertar(lista-nodos, frontera)

Evaluación de estrategias de búsqueda I

- Criterios para la evaluación de estrategias de búsqueda
  - Completitud: ¿está garantizado que el algoritmo encuentre una solución si la hay?
  - Optimalidad: ¿encuentra la solución de coste mínimo?
  - Complejidad en tiempo: ¿cuánto tarda en encontrar una solución en el peor caso (cuando el nodo objetivo es el último del árbol)? → número de nodos expandidos
  - Complejidad en espacio: ¿cuánta memoria se necesita en el peor caso? (máximo número de nodos simultáneamente en memoria)
  - Complejidad
    - r: factor de ramificación. Máximo número de hijos de un nodo.
    - p: profundidad del nodo objetivo más superficial.
    - m: longitud máxima de cualquier camino en el espacio de estados.

Evaluación de estrategias de búsqueda II

- Coste total = coste solución + coste búsqueda
  - Coste solución: suma del coste de los operadores usados en el camino.
  - Coste búsqueda: complejidad del algoritmo.
- Hay que llegar a un compromiso entre ambos costes → Obtener la mejor solución posible con los recursos disponibles
  - Puede que no interese una solución óptima que requiera un coste excesivo.

Tipos de estrategias de búsqueda

- Estrategias de búsqueda no informada o a ciegas:
  - Exploración exhaustiva del espacio de búsqueda hasta encontrar una solución.
  - No incorporan conocimiento que guíe la búsqueda. Se decide a priori qué camino sigue.
  - La búsqueda no incorpora información del dominio.
- Estrategias de búsqueda informada o heurística:
  - Exploración de los caminos "más prometedores".
  - Se incorpora conocimiento del dominio → Funciones Heurísticas.
  - Las Funciones Heurísticas son "pistas" para acotar el proceso de búsqueda y hacerlo más eficiente.

Estrategias de búsqueda no informada

- Búsqueda primero en anchura.
- Búsqueda de coste uniforme.
- Búsqueda primero en profundidad.
- Búsqueda de profundidad limitada.
- Búsqueda primero en profundidad con profundidad iterativa.
- Búsqueda bidireccional.

Búsqueda primero en anchura

- Se explora el espacio de búsqueda haciendo un recorrido por niveles.
- Un nodo se visita solamente cuando todos sus predecesores y sus hermanos anteriores en orden de generación ya se han visitado.
- Primero se expande el nodo raíz, después se expanden todos los sucesores de este nodo, después los sucesores del siguiente, …
- Nodos abiertos del algoritmo general se implementa con una cola → Para visitar los nodos en el orden establecido por el algoritmo de búsqueda.

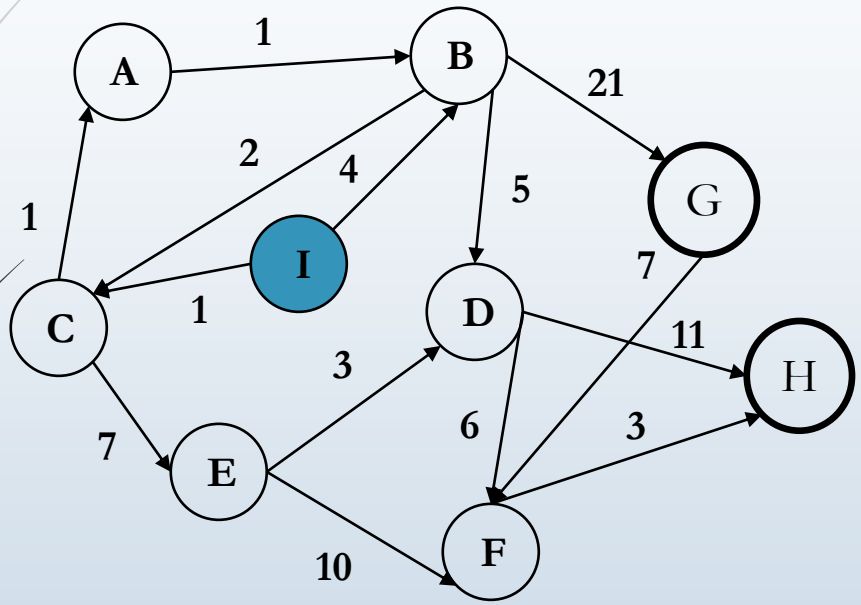
Búsqueda primero en anchura. Evaluación

- Es un algoritmo completo → nos garantiza llegar al nivel donde se encuentra la solución (si la hay)
- Complejidad en tiempo: $O(r^{p+1})$
  - Suponiendo un factor de ramificación máximo r y un camino hasta la solución de profundidad p
  - $r^0 + r^1 + r^2 + \ldots + r^p + (r^{p+1} - r) = O(r^{p+1}) \approx O(r^p)$
- Complejidad en espacio: $O(r^{p+1}) \approx O(r^p)$
  - Todos los nodos abiertos (pendientes de explorar) han de mantenerse en memoria.
  - Sólo viable para casos pequeños.
- Si los operadores de búsqueda tienen coste uniforme la solución obtenida es óptima respecto al número de pasos desde la raíz.
- Los problemas de búsqueda con complejidad exponencial no pueden resolverse con estrategias sin información.

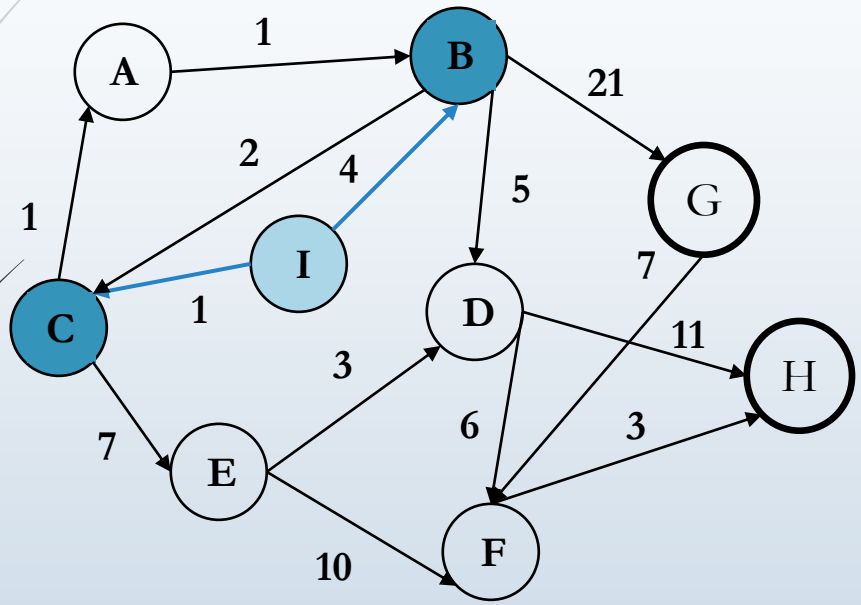
Búsqueda primero en anchura. Nodos repetidos

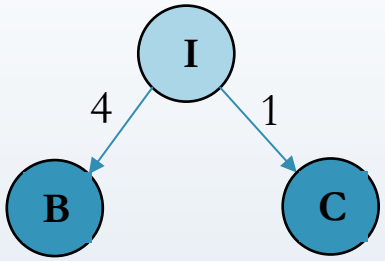
- Si el nodo generado actual está repetido en niveles superiores, su coste será peor ya que su camino desde la raíz es más largo.
- Si está repetido al mismo nivel, su coste será el mismo.
- Conclusión: para cualquier nodo repetido, su coste será peor o igual que algún nodo anterior visitado o no, de manera que lo podremos descartar, ya que o lo hemos expandido ya o lo haremos próximamente.
- No hacer el tratamiento de nodos repetidos no supone ninguna ganancia, ya que tanto la cola de nodos abiertos como la de nodos repetidos crecen exponencialmente.

Búsqueda primero en anchura. Ejemplo

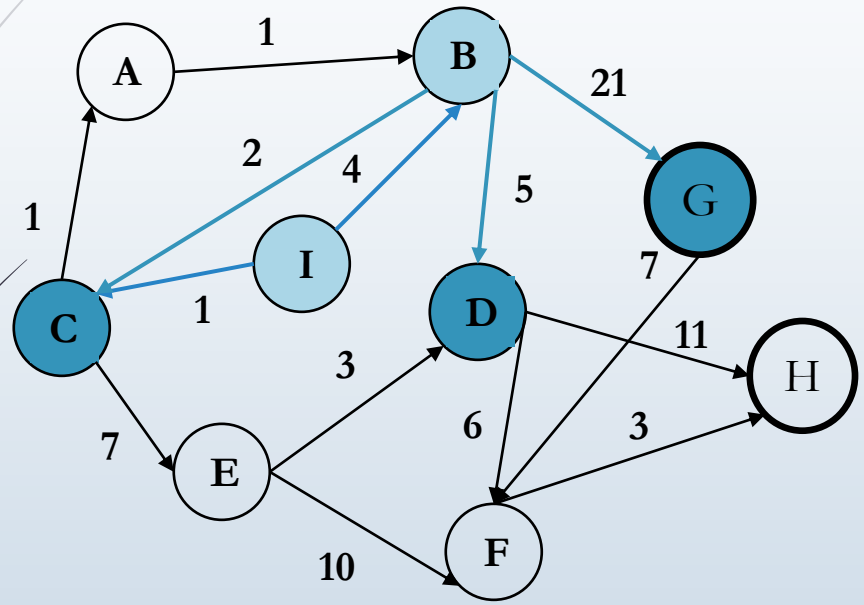


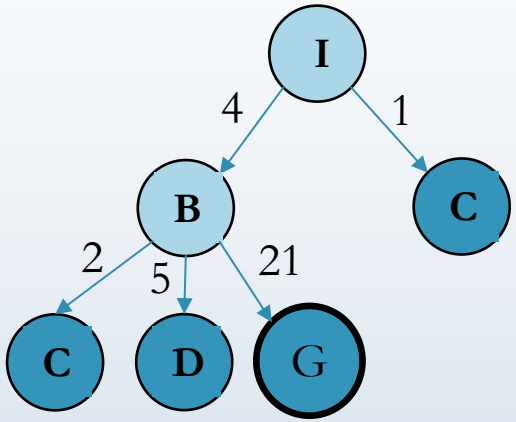
Búsqueda primero en anchura. Ejemplo



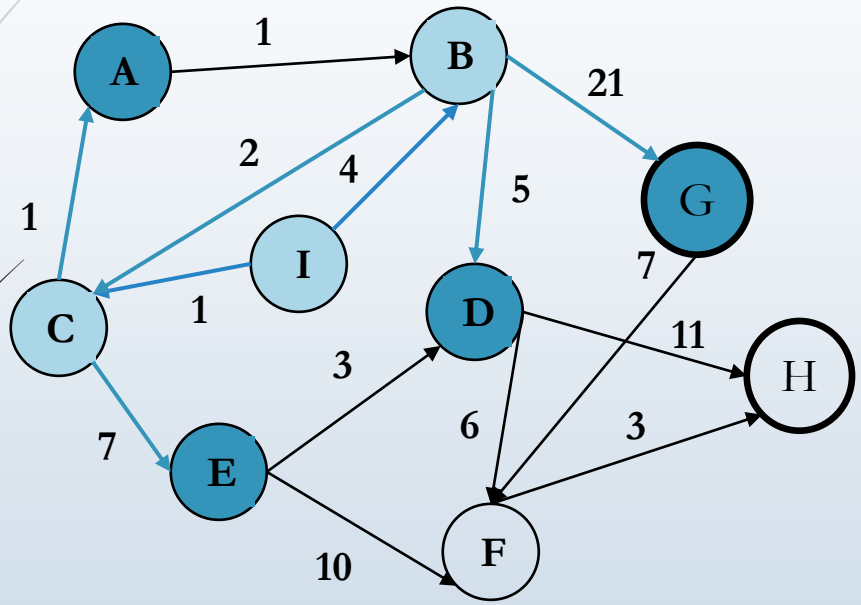


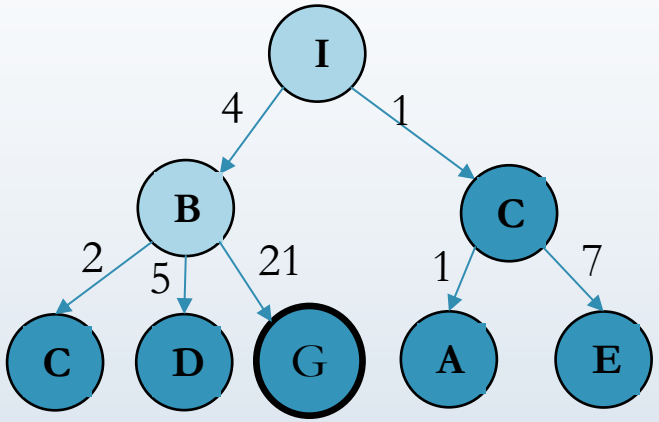
Búsqueda primero en anchura. Ejemplo



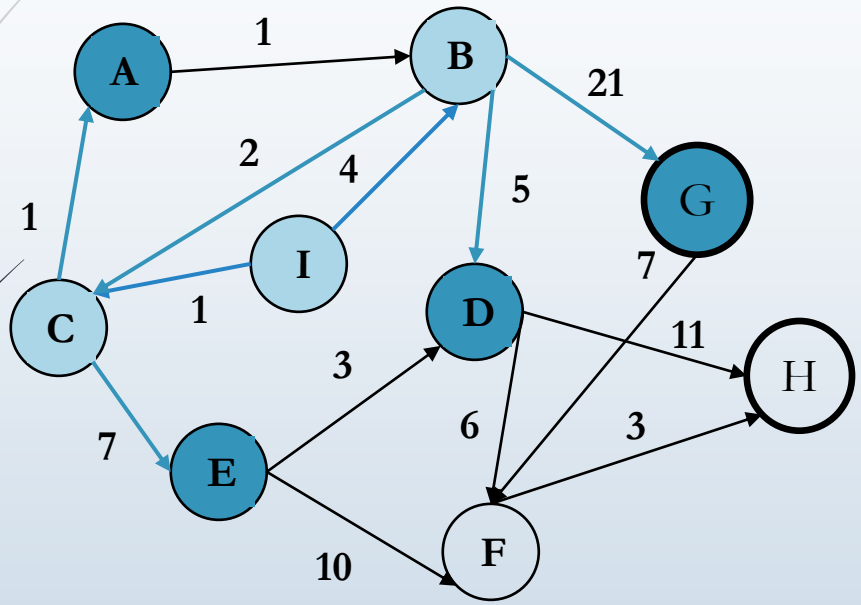


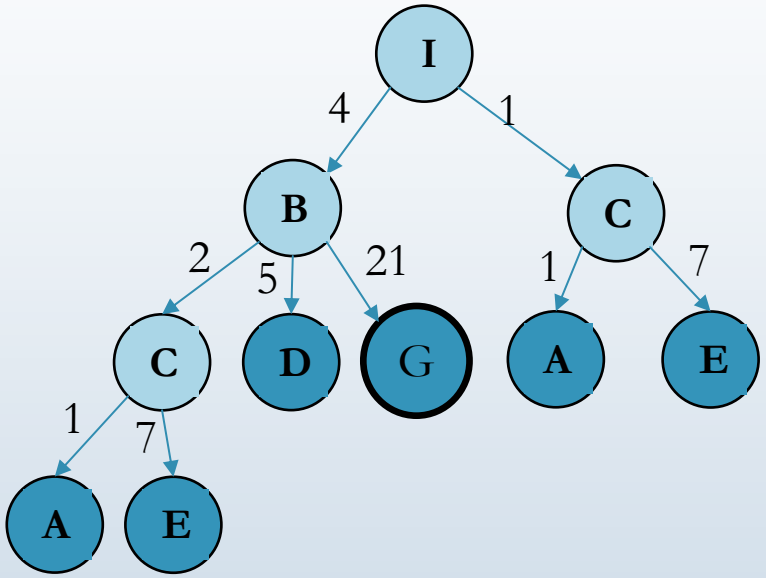
Búsqueda primero en anchura. Ejemplo



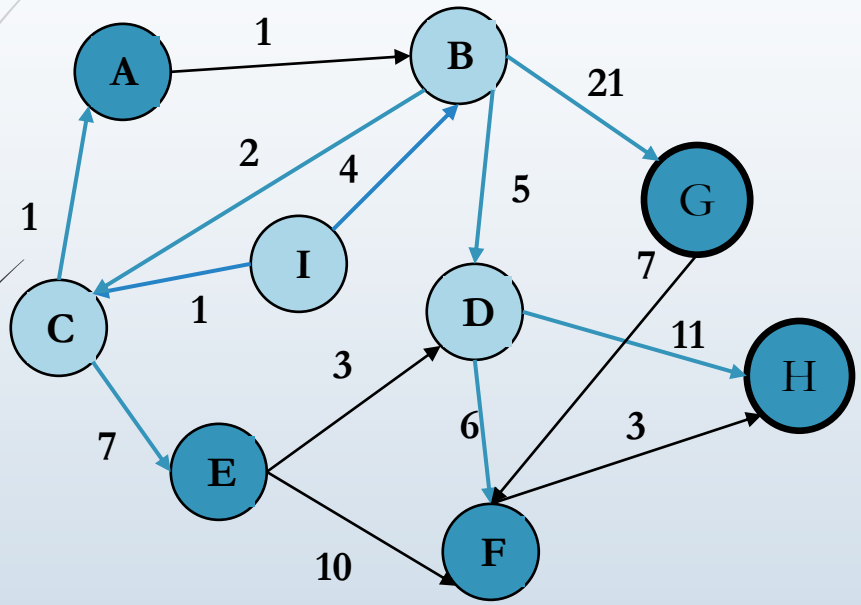


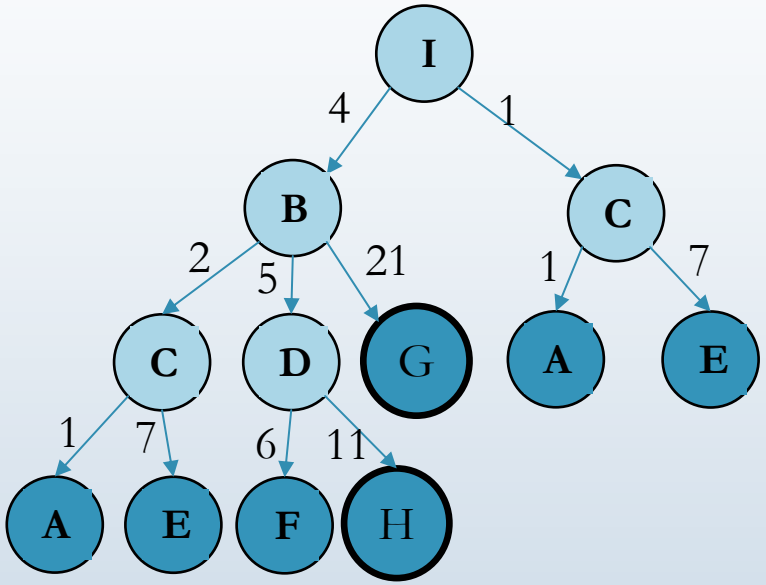
Búsqueda primero en anchura. Ejemplo



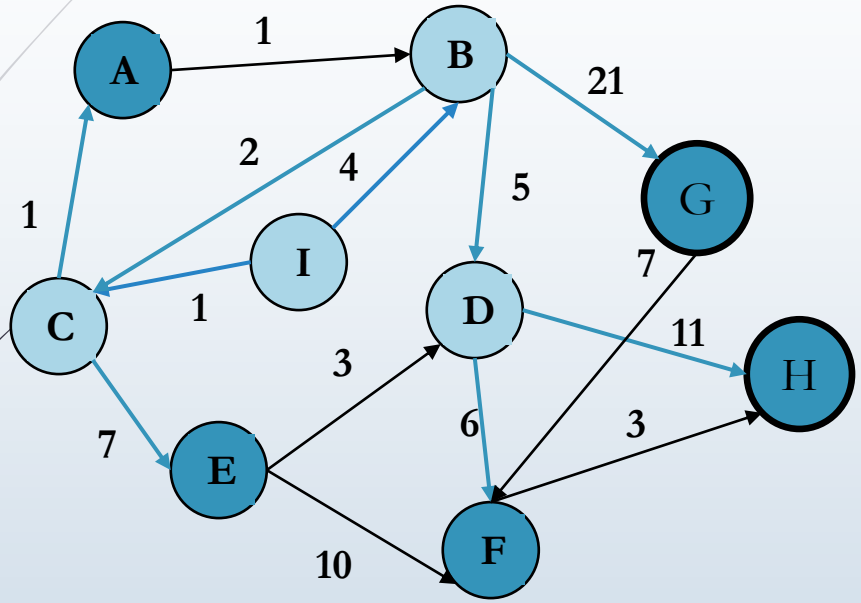


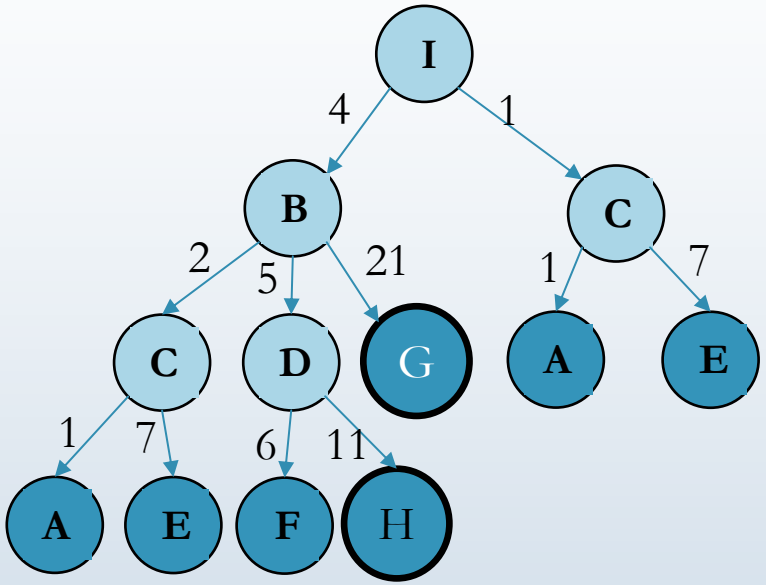
Búsqueda primero en anchura. Ejemplo





Búsqueda primero en anchura. Ejemplo





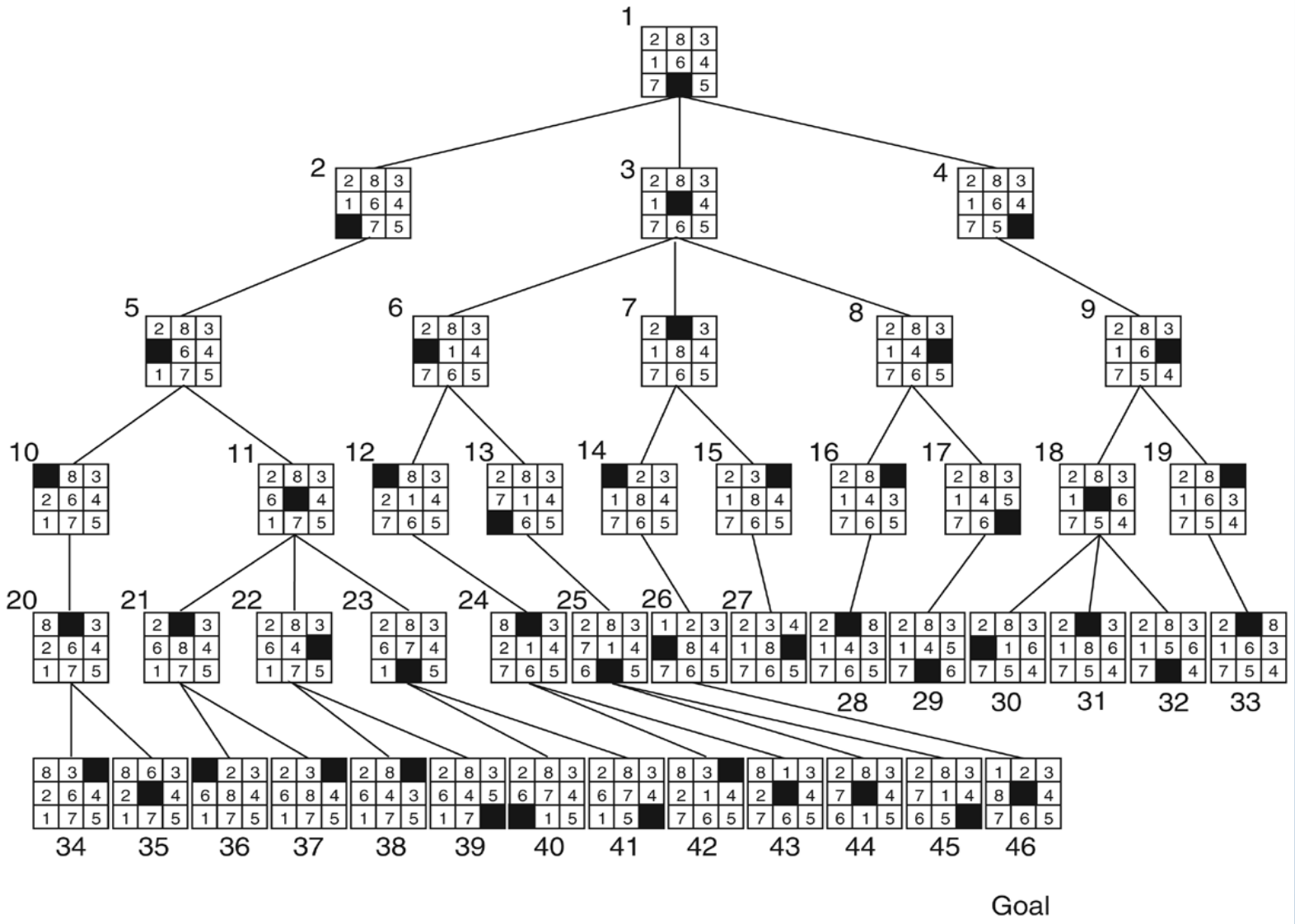
Nodos expandidos: I B C C D G

Nodos generados: I B C C D G A E A E F H

Camino a la solución: I B G

Coste: 4 + 21 = 25

Búsqueda primero en anchura. 8-puzzle



Búsqueda primero en anchura. Ventajas

- Es más difícil que quede atrapada explorando callejones sin salida.
- Si existe una solución garantiza que la encuentra y además será una de las de longitud mínima.

Preguntas sobre búsqueda

- Kahoot
- www.kahoot.it

Búsqueda de coste uniforme

- La búsqueda primero en anchura es óptima cuando todos los costes son iguales.
- En cada paso, de los nodos en abiertos, se expande el nodo que tiene el menor coste del camino hasta llegar a él.
- Si hay varios iguales se elige aleatoriamente o de izquierda a derecha.
- Abiertos se implementa con una cola de prioridad.
- Si el coste del camino a un nodo coincide con su profundidad (o es directamente proporcional a la profundidad) entonces esta búsqueda equivale a primero en anchura.
- Si todos los costes son iguales es equivalente a la búsqueda primero en anchura.

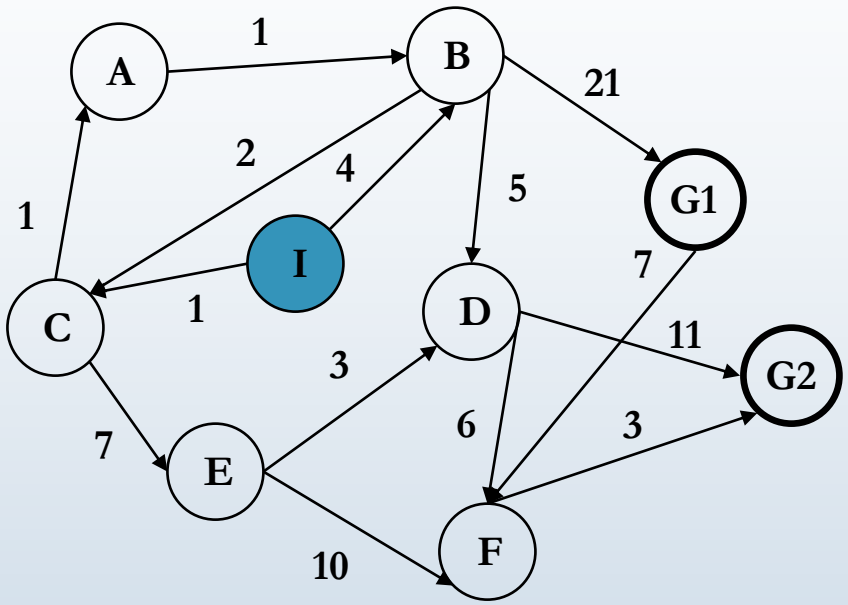
Búsqueda de coste uniforme. Evaluación

- Completa: SÍ, si no existen caminos infinitos de coste finito.
  - Si el coste de cada paso es mayor o igual a una constante positiva pequeña ε
- Óptima: SÍ, si coste(sucesor(n)) >= coste(n) → Se satisface cuando todos los operadores tienen coste > 0
- Complejidad en espacio y tiempo:
  - equivalente a primero en anchura cuando todos los costes son iguales: $O(r^p)$.
  - Si definimos C* como el coste de la solución óptima: $O(r^{C^*/\varepsilon})$.

Búsqueda de coste uniforme. Nodos repetidos

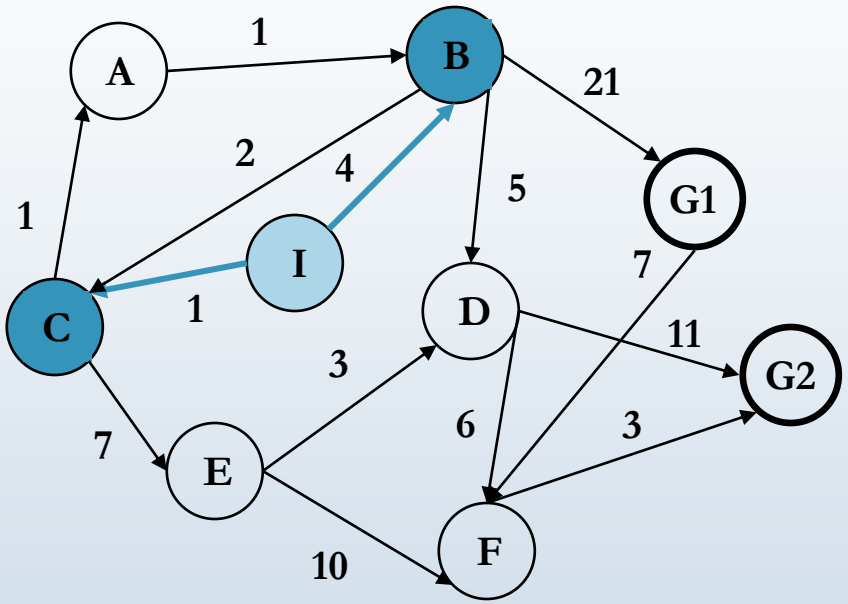
- El problema de las repeticiones es mucho mayor.
- Un control indiscriminado de repeticiones supondría que la estrategia dejaría de ser óptima...
  - Puede verse en el siguiente ejemplo (el B hijo de A no se generaría y no se encontraría esta solución óptima)

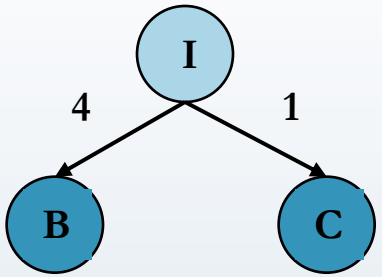
Búsqueda de coste uniforme. Ejemplo



Cola de nodos abiertos: I(0)

Búsqueda de coste uniforme. Ejemplo

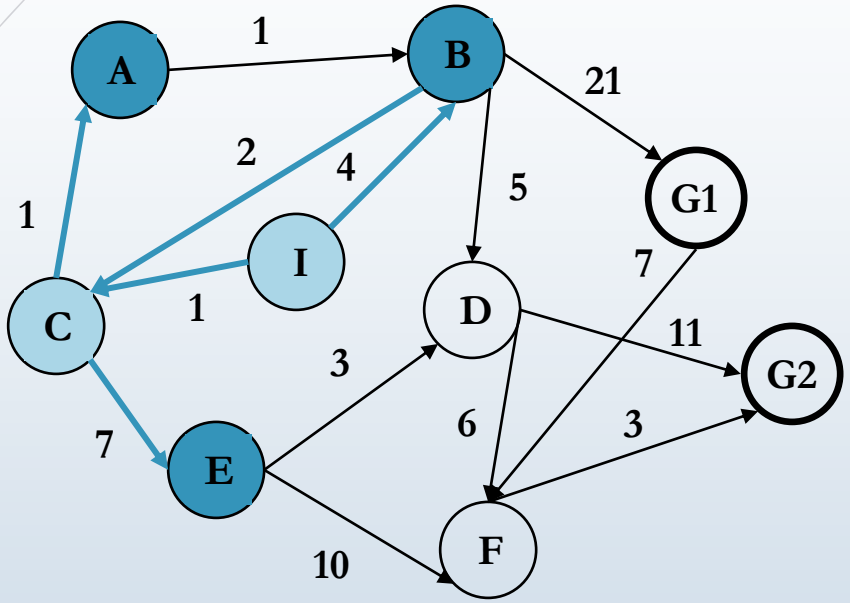


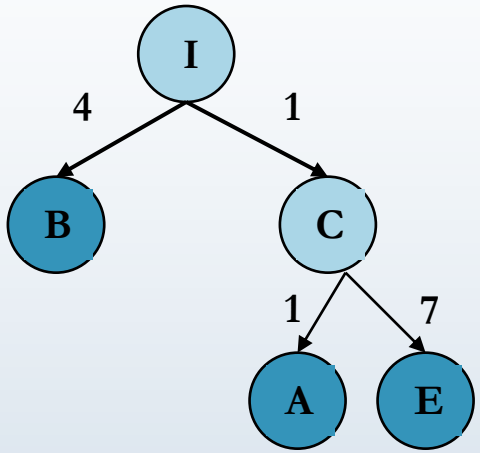


Nodos abiertos: B(4) C(1)

Nodos cerrados: I

Búsqueda de coste uniforme. Ejemplo

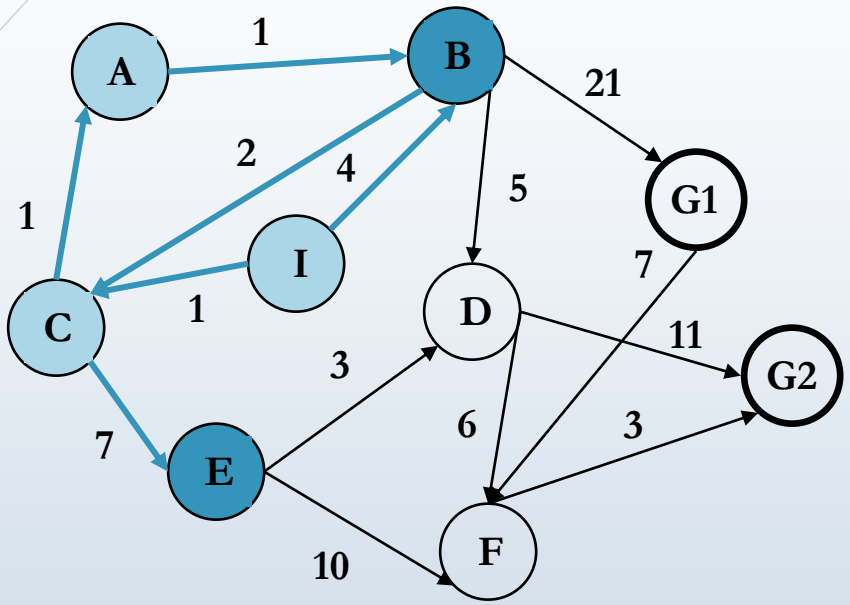


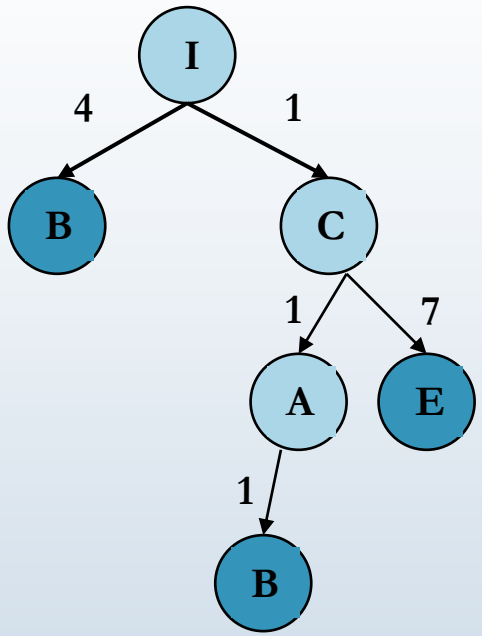


Nodos abiertos: E(8) B(4) A(2)

Nodos cerrados: I C

Búsqueda de coste uniforme. Ejemplo

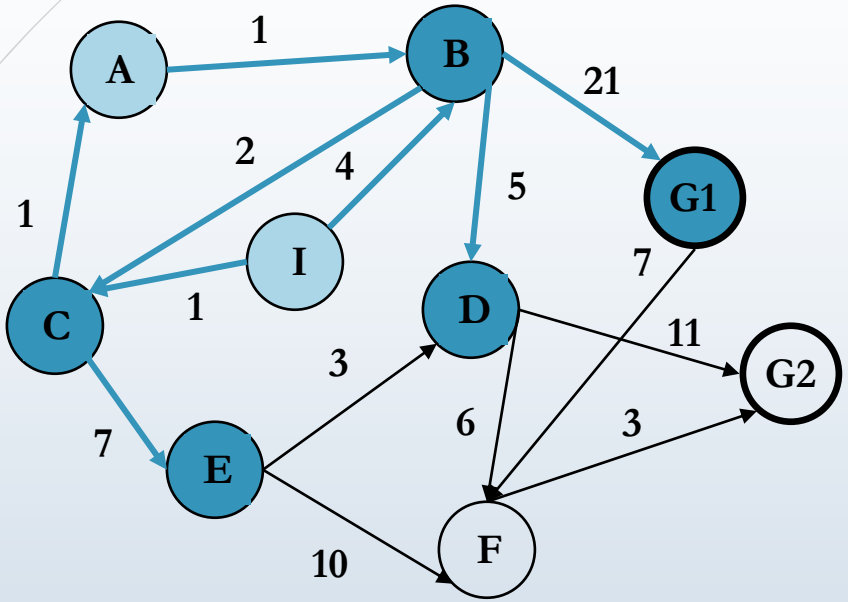


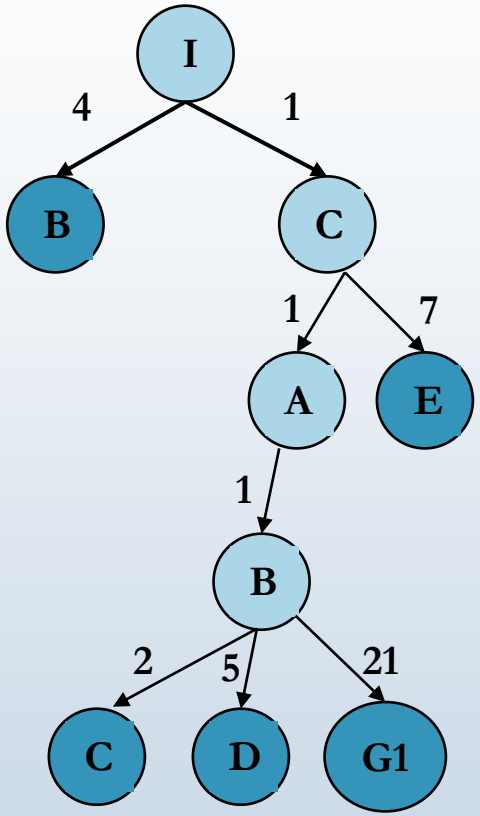


Nodos abiertos: E(8) B(4) B(3)

Nodos cerrados: I C A

Búsqueda de coste uniforme. Ejemplo

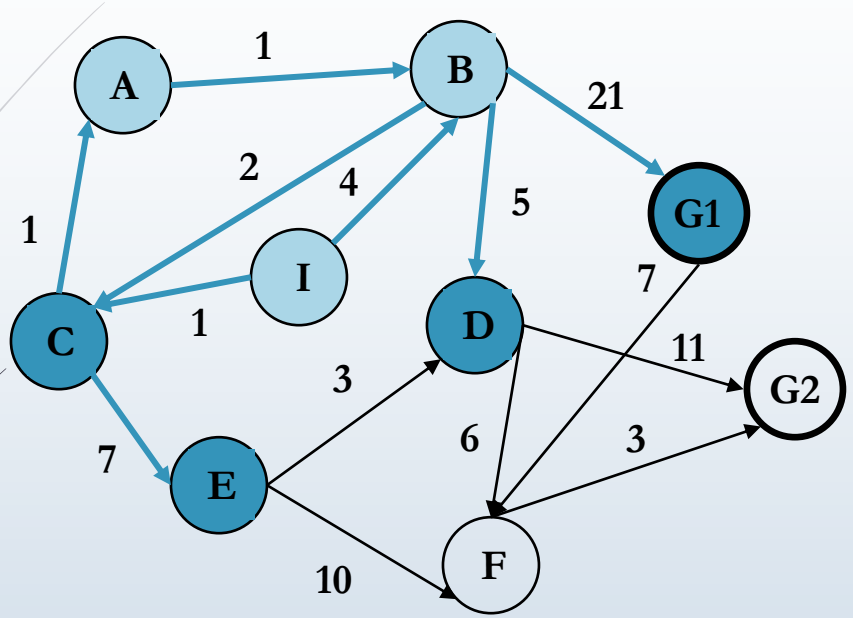


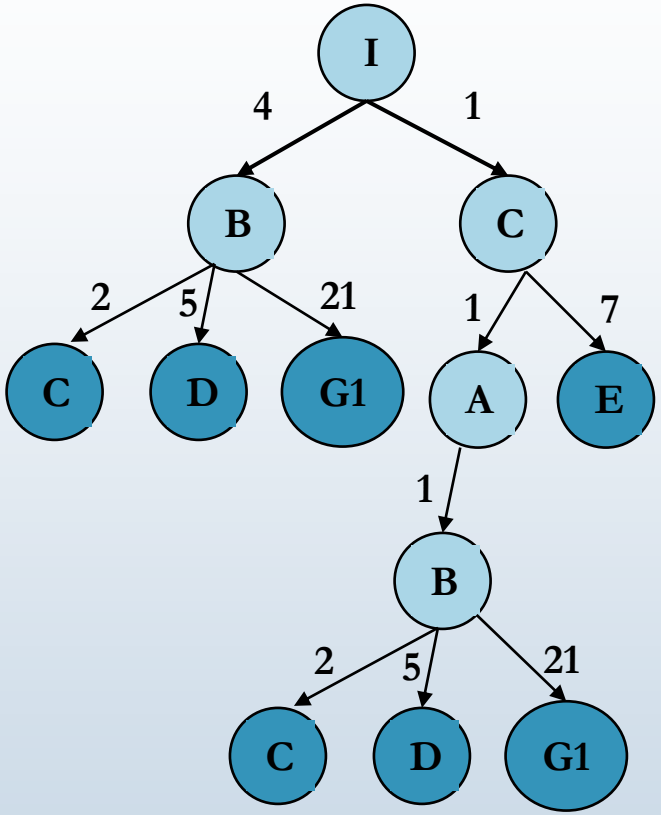


Nodos abiertos: G1(24) D(8) E(8) C(5) B(4)

Nodos cerrados: I A

Búsqueda de coste uniforme. Ejemplo

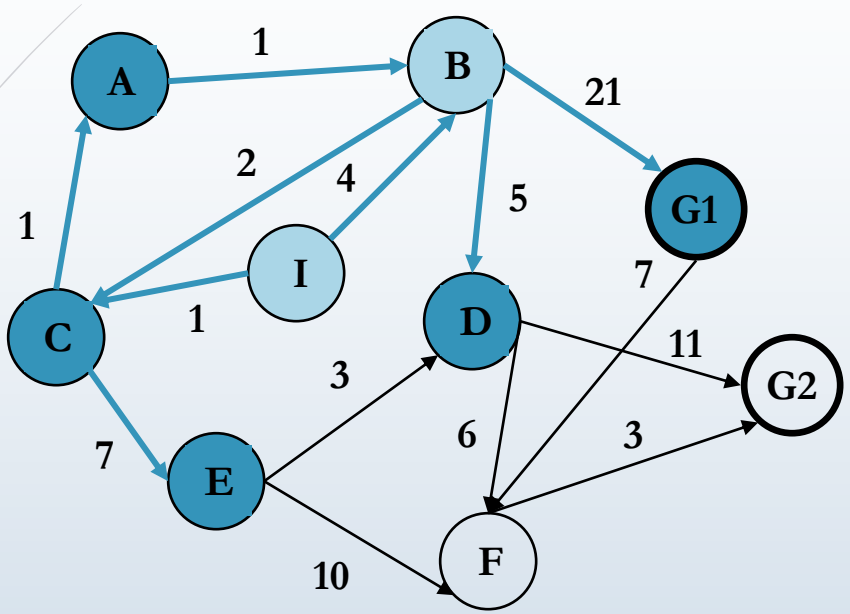


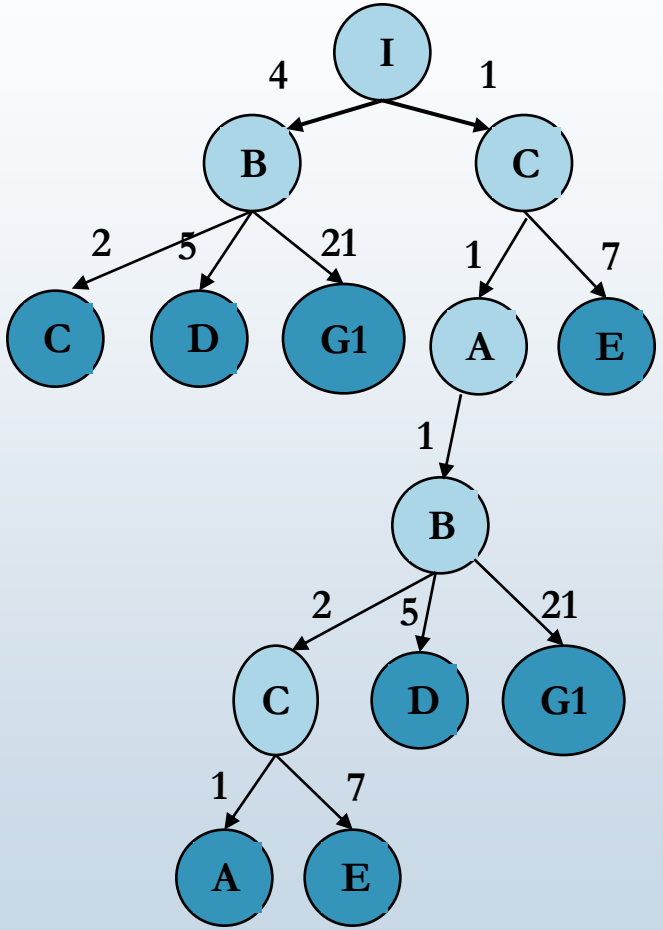


Nodos abiertos: G1(25) G1(24) D(9) D(8) E(8) C(6) C(5)

Nodos cerrados: I A B

Búsqueda de coste uniforme. Ejemplo

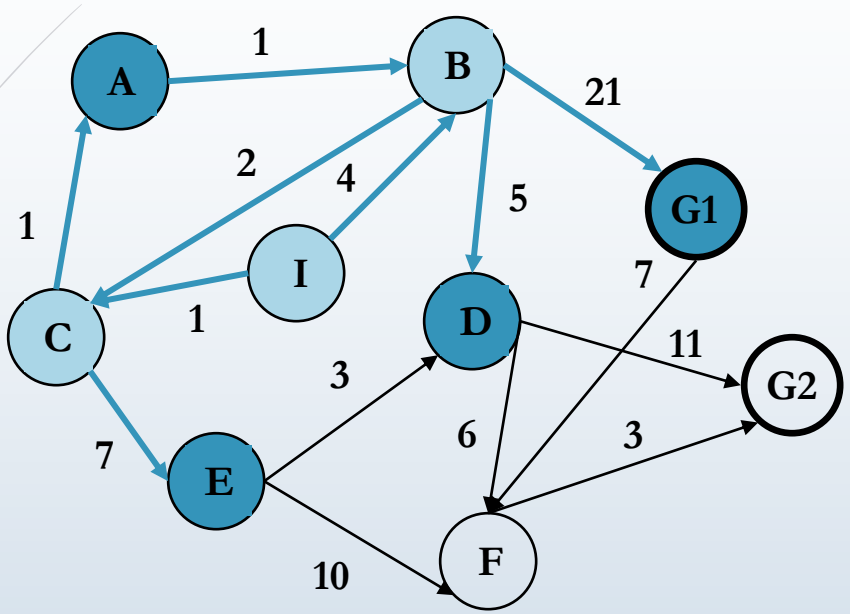


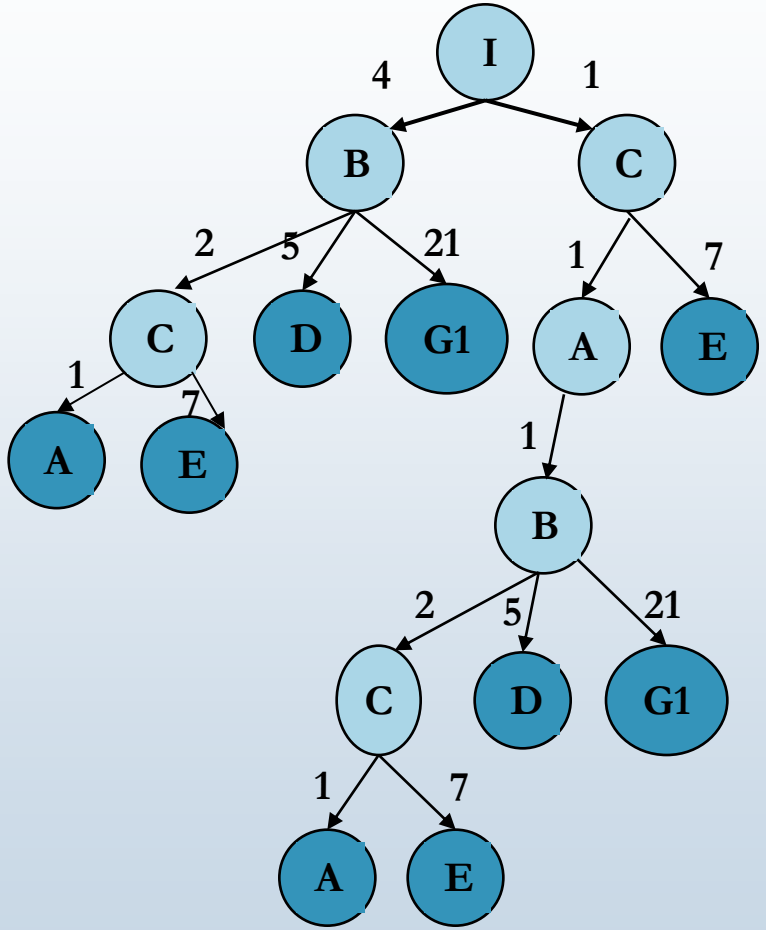


Nodos abiertos: G1(25) G1(24) E(12) D(9) D(8) E(8) A(6) C(6)

Nodos cerrados: I B

Búsqueda de coste uniforme. Ejemplo

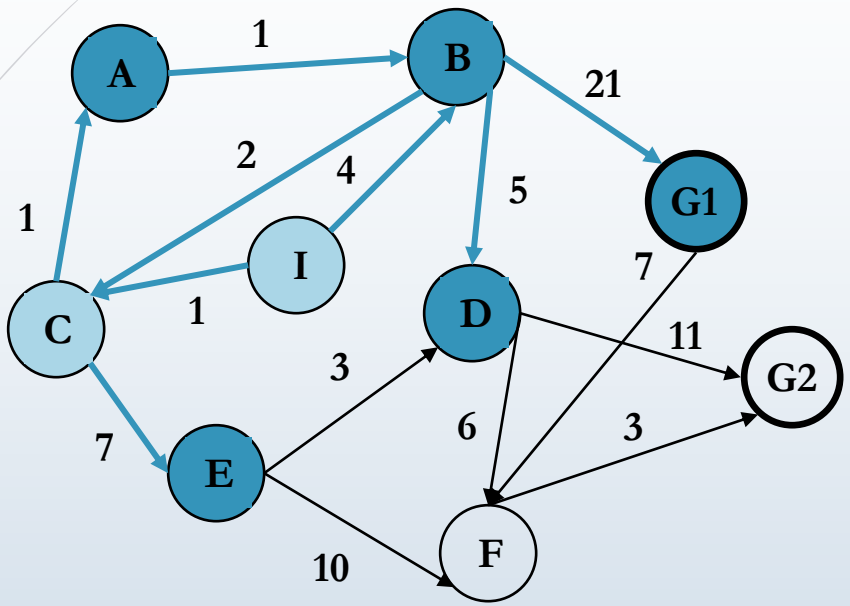


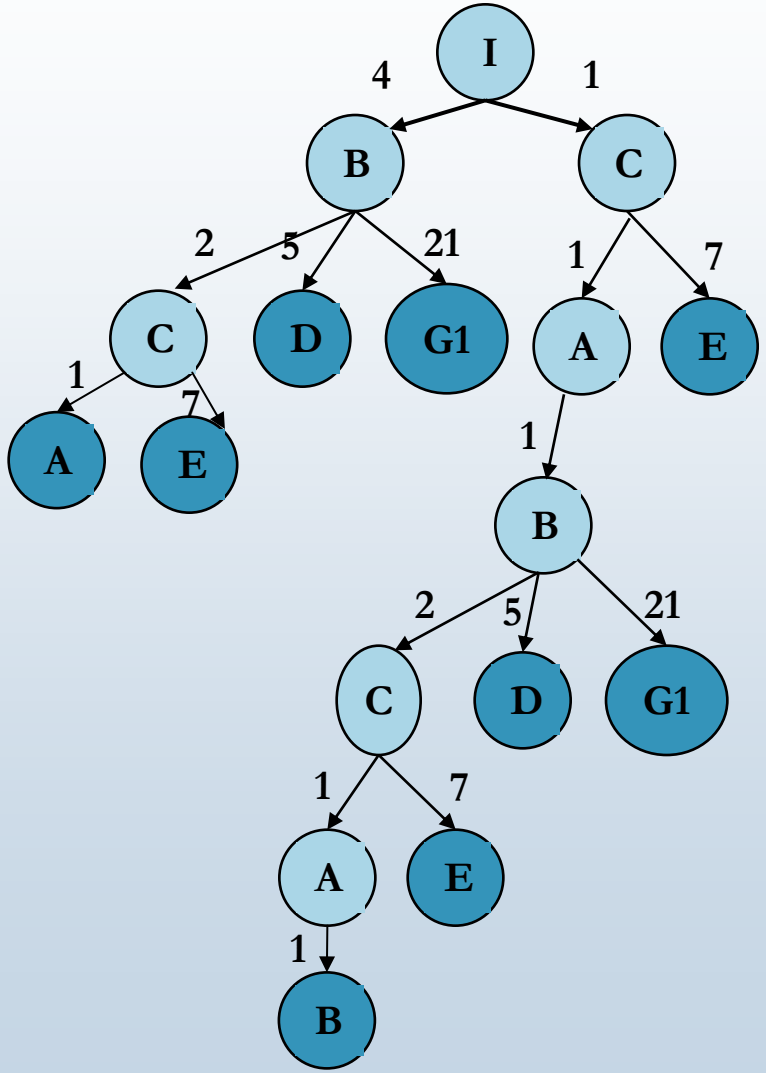


Nodos abiertos: G1(25) G1(24) E(13) E(12) D(9) D(8) E(8) A(7) A(6)

Nodos cerrados: I B C

Búsqueda de coste uniforme. Ejemplo

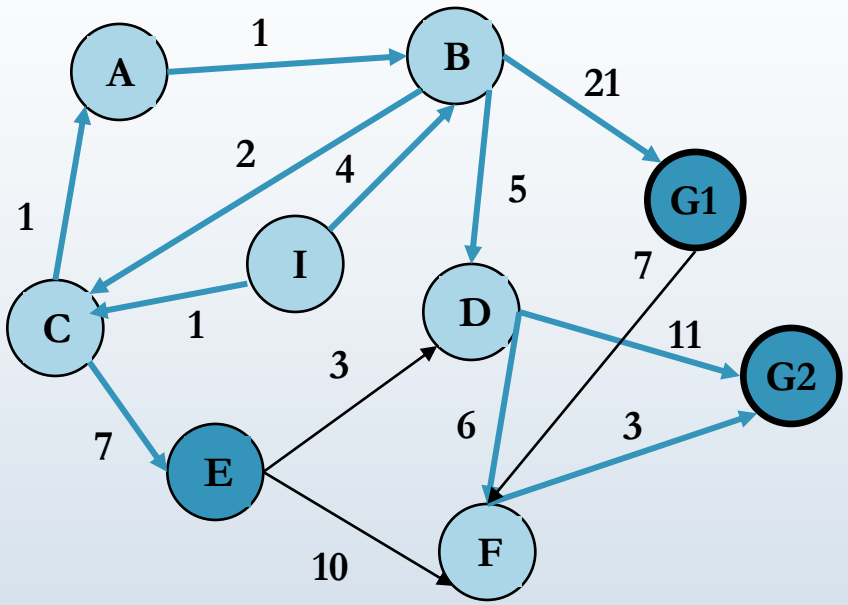




Nodos abiertos: G1(25) G1(24) E(13) E(12) D(9) D(8) E(8) B(7) A(7)

Nodos cerrados: I C

Búsqueda de coste uniforme. Ejemplo

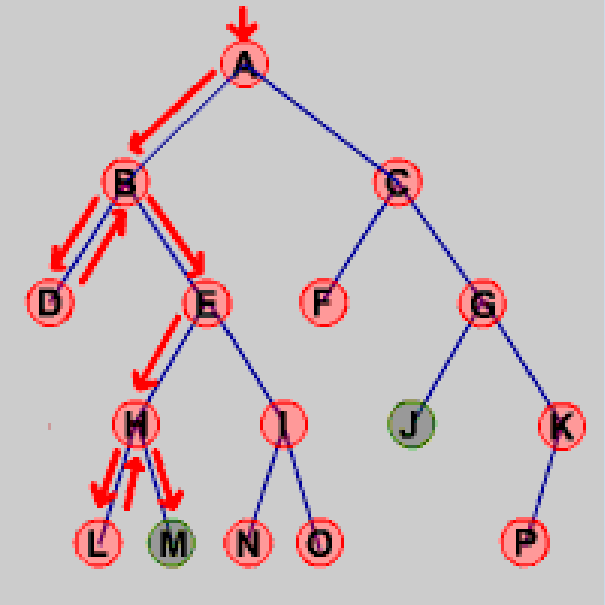


Camino a la solución: I C A B D F G2

Coste: 1+1+1+5+6+3 = 17

Búsqueda primero en profundidad

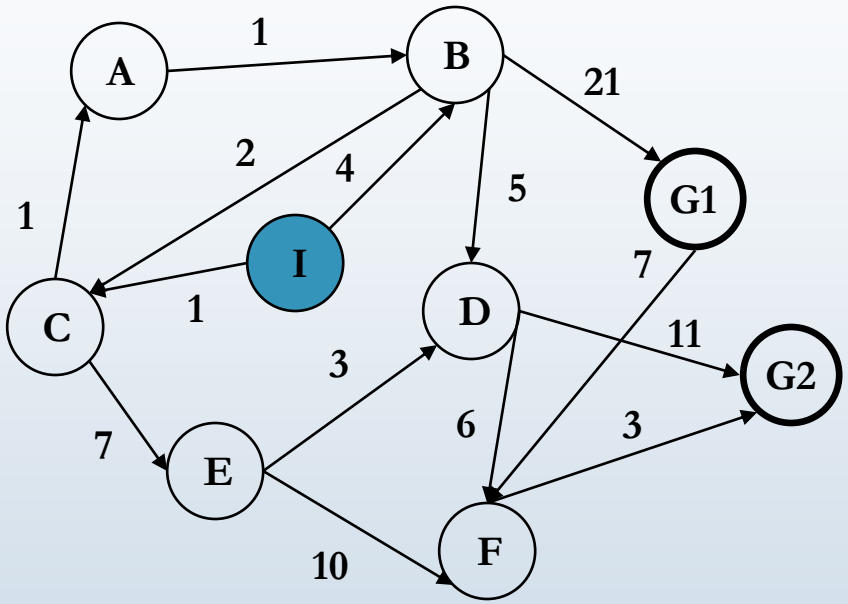
- Los nodos se expanden por orden inverso de generación (cola LIFO = pila).
- Expande el nodo más profundo en la frontera.
- En cada paso:
  - El nodo actual es el último que entró en abiertos.
  - Si el nodo actual no tiene sucesores y no es objetivo → se quita de abiertos. Se da marcha atrás y se expande el siguiente nodo más profundo.
  - En otro caso: se expande (y se quita de abiertos).
- Abiertos se implementa con una pila (o se usa recursión).



Búsqueda primero en profundidad. Evaluación

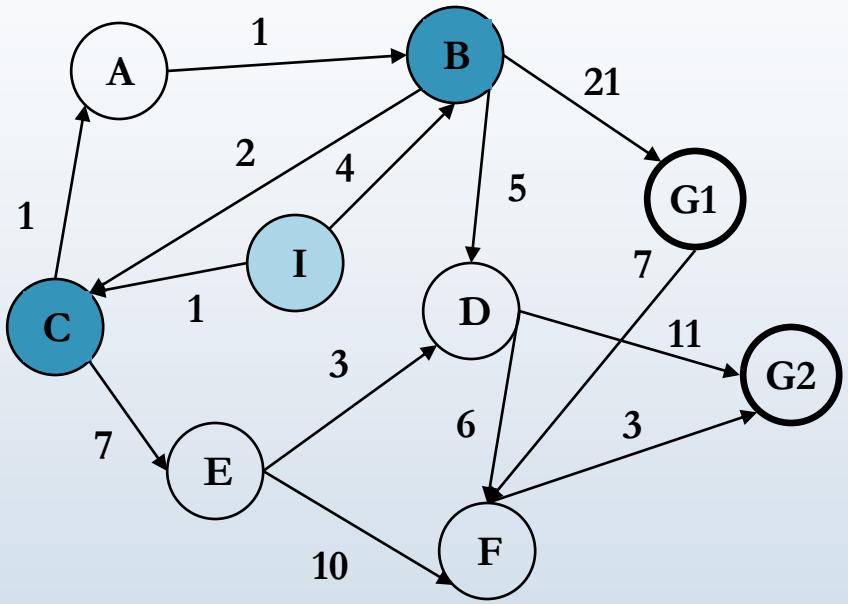
- Completa: NO; puede meterse en caminos infinitos.
- Óptima: NO; puede haber soluciones mejores por otros caminos.
  - No recomendable si la máxima profundidad es grande.
- Complejidad Espacio: O(r*m), siendo r el factor de ramificación y m la profundidad máxima.
  - Basta guardar el camino actual y los nodos no expandidos.
- Complejidad Tiempo: $O(r^m)$
  - Si hay muchas soluciones, puede ser más rápida que primero en anchura; depende del orden de aplicación de operadores

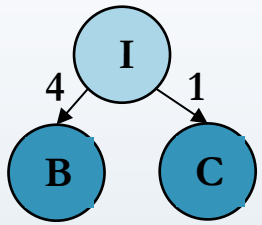
Búsqueda primero en profundidad. Ejemplo



Pila de nodos abiertos: I

Búsqueda primero en profundidad. Ejemplo

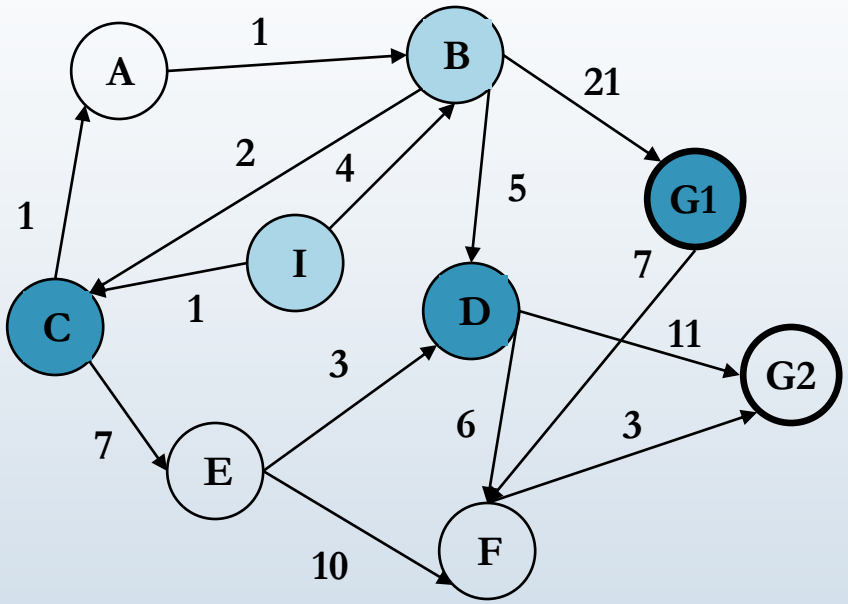


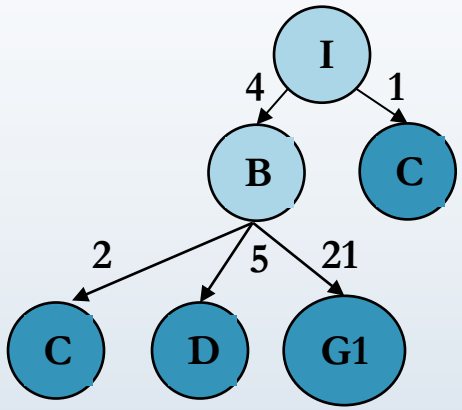


Pila de nodos abiertos: B C

Nodos cerrados: I

Búsqueda primero en profundidad. Ejemplo

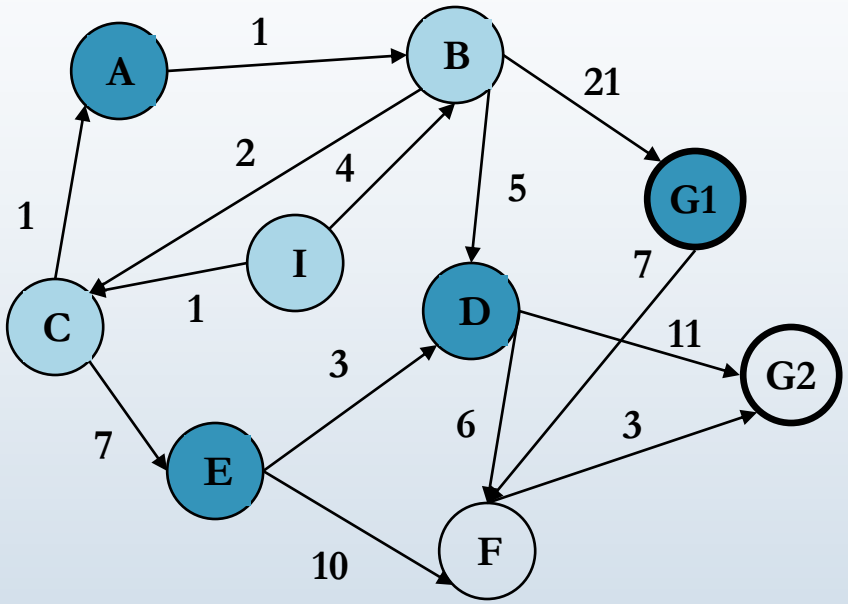


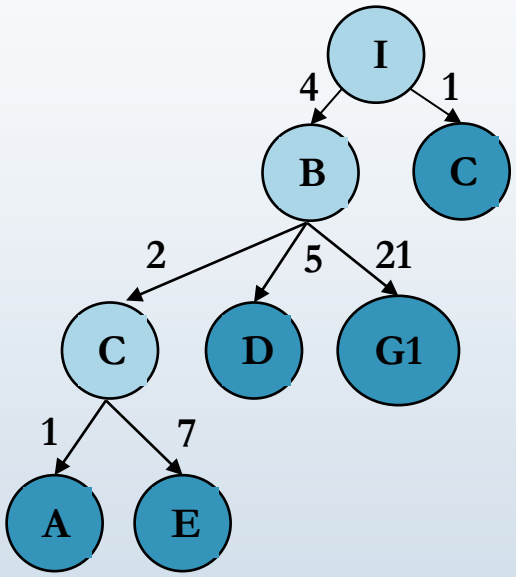


Pila de nodos abiertos: C D G1 C

Nodos cerrados: I B

Búsqueda primero en profundidad. Ejemplo

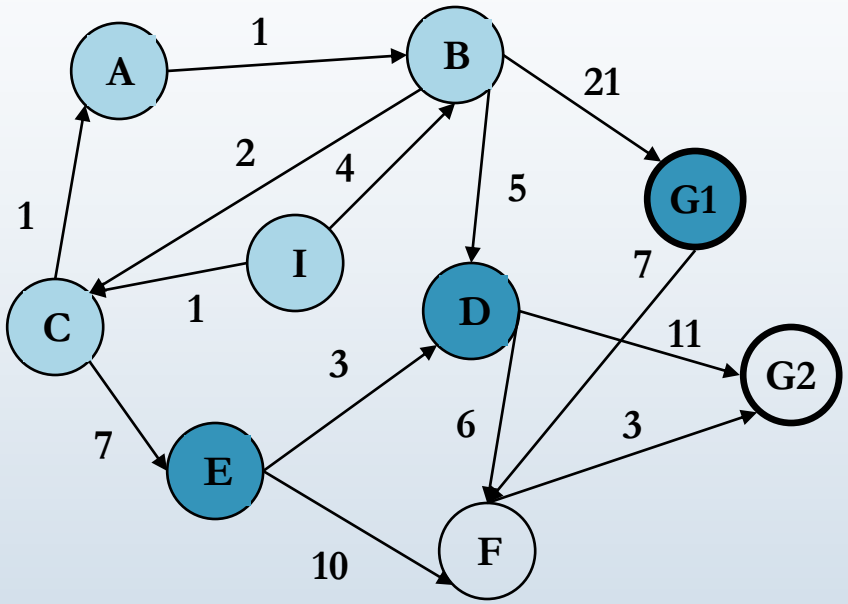


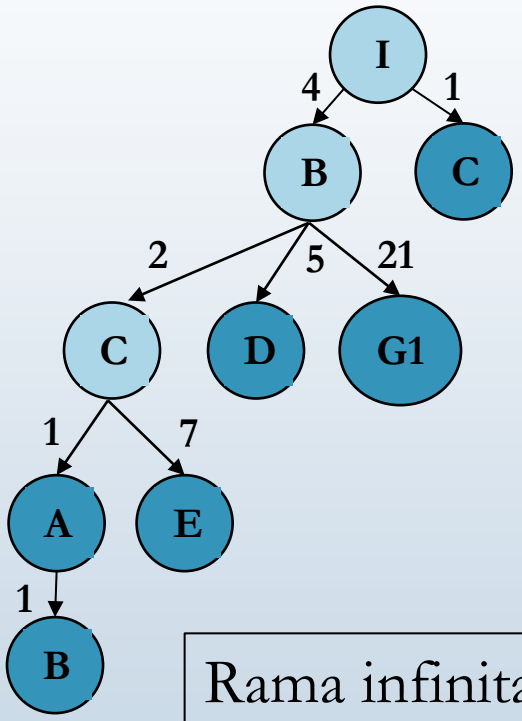


Pila de nodos abiertos: A E D G1 C

Nodos cerrados: I B C

Búsqueda primero en profundidad. Ejemplo



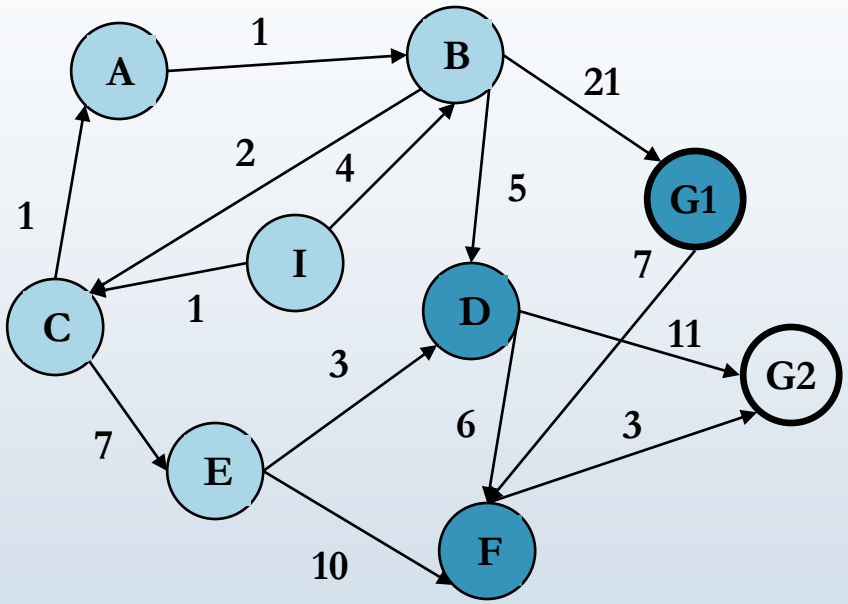


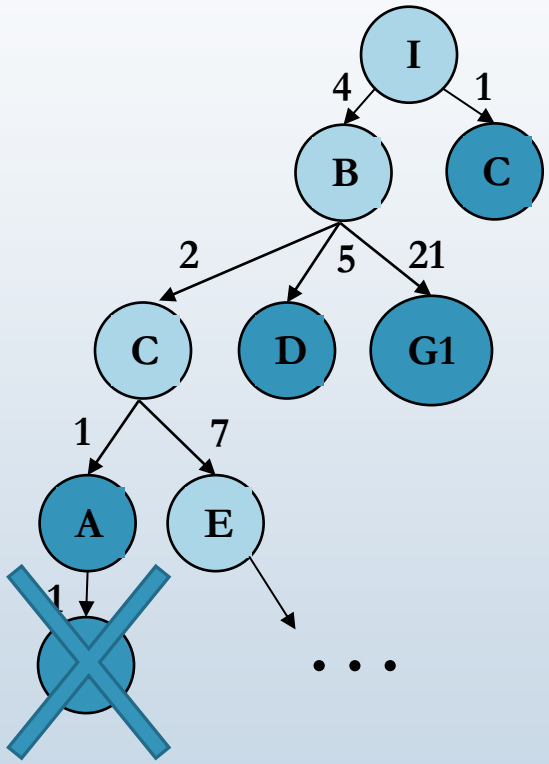
Pila de nodos abiertos: B E D G1 C

Nodos cerrados: I B C A

Rama infinita.
Sin control de ciclos no encuentra la solución

Búsqueda primero en profundidad. Ejemplo

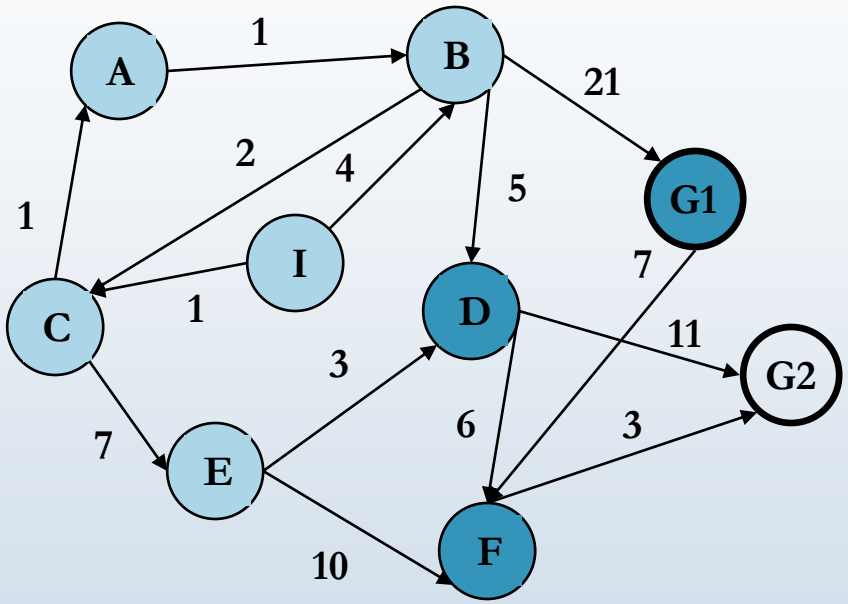


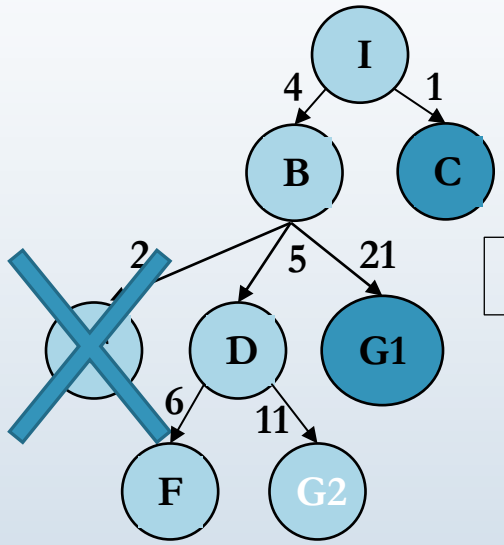


Nodos cerrados: I B C A E D F G2

Solución: I B C A E D F G2 Coste: 4+2+7+3+6+3 = 25

Búsqueda primero en profundidad. Ejemplo





Si miramos mejor

Nodos cerrados: I B D F G2

Nodos abiertos: I B C D G1 F G2

Solución: I B D G2 Coste: 4+5+11 = 20

Búsqueda primero en profundidad. Ventajas

- Necesita menos memoria.
  - Sólo se almacenan nodos del camino que se sigue, mientras que en anchura se almacena la mayor parte del árbol que se va generando.
- Con "suerte" puede encontrar una solución sin tener que examinar gran parte del espacio de estados (depende en gran medida de que el orden de las reglas sea "adecuado“).
  - Es muy dependiente del orden de aplicación de las reglas.

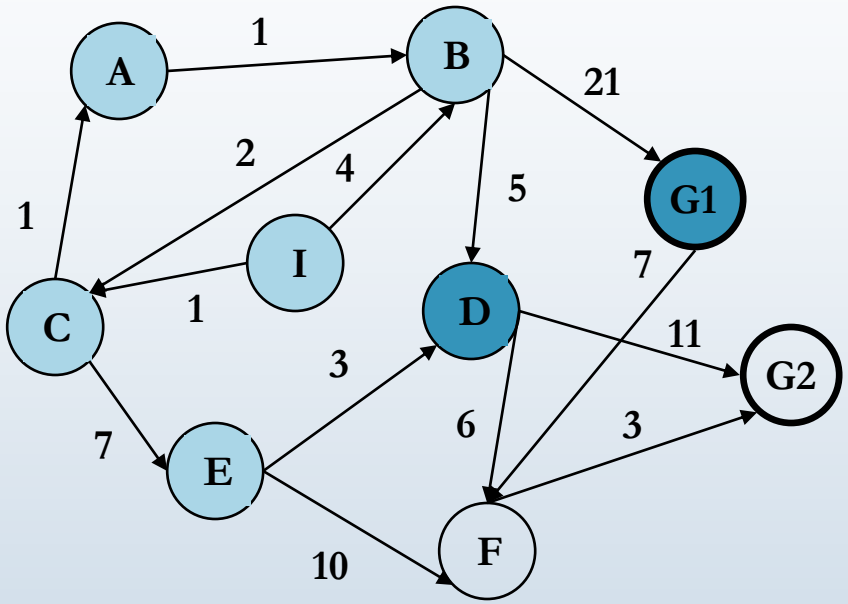
Búsqueda de profundidad limitada

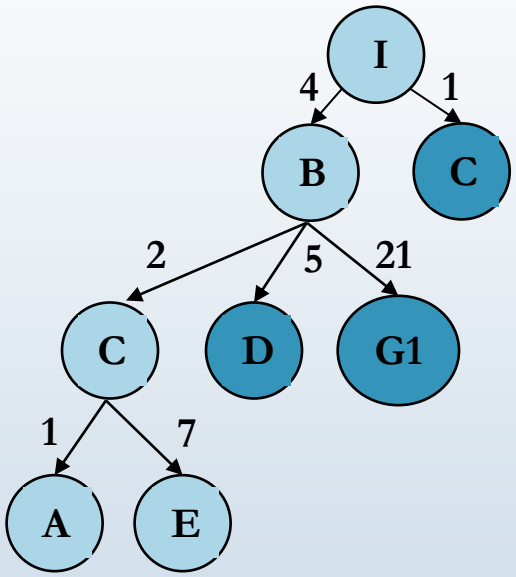
- Es la búsqueda en profundidad con límite L de profundidad para evitar descender indefinidamente por el mismo camino.
- El límite permite desechar caminos en los que se supone que no encontraremos un nodo objetivo lo suficientemente cercano al nodo inicial.

Búsqueda de profundidad limitada. Evaluación

- Completa sólo si L >= p (p profundidad mínima de "la solución óptima")
  - Si p es desconocido, la elección de L es una incógnita.
- No óptima → No puede garantizarse que la primera solución encontrada sea la mejor.
- Tiempo: $O(r^L)$
- Espacio: $O(r*L)$
- Si $L = \infty$ => búsqueda primero en profundidad

Búsqueda de profundidad limitada. Ejemplo L=3

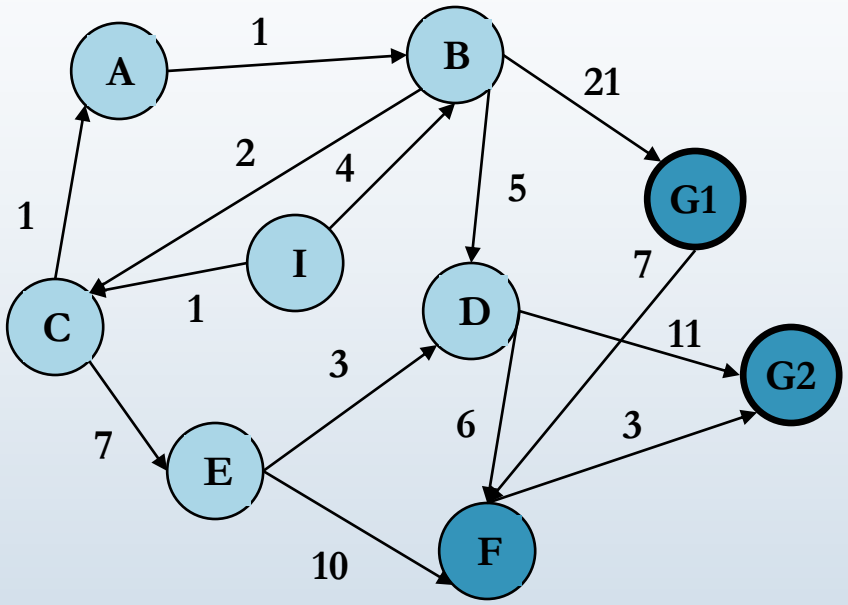


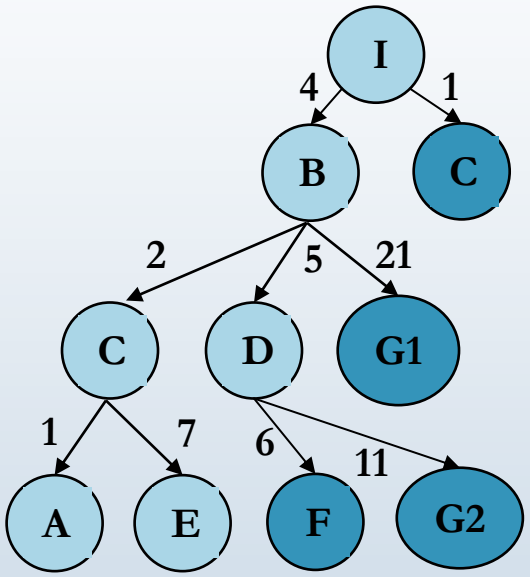


Pila de nodos abiertos: D G1 C

Nodos cerrados: I B C A E

Búsqueda de profundidad limitada. Ejemplo L=3

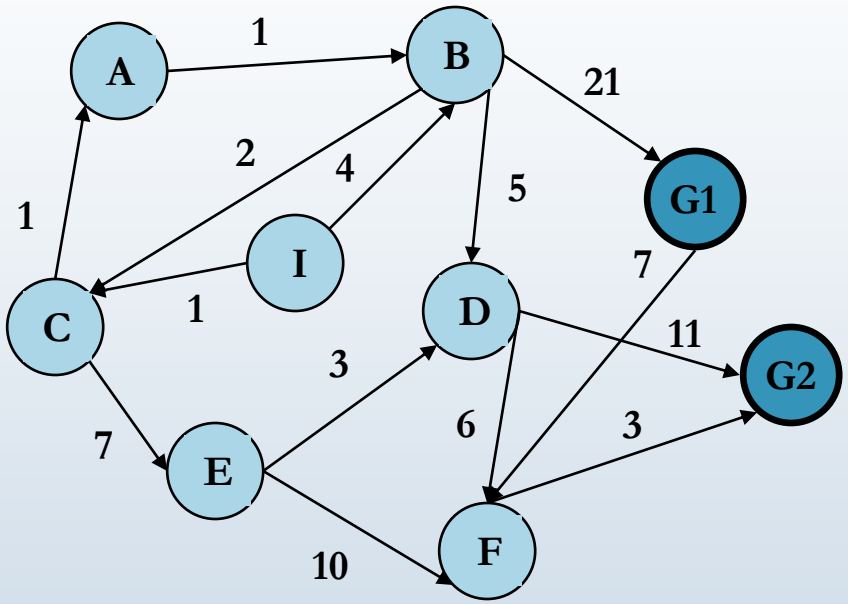


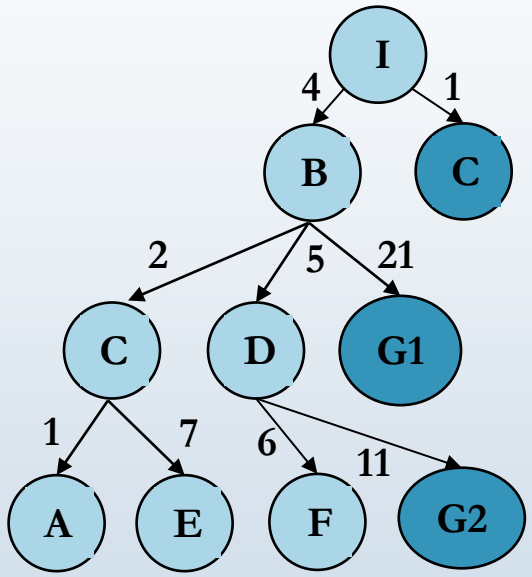


Pila de nodos abiertos: F G2 D G1 C

Nodos cerrados: I B C A E D

Búsqueda de profundidad limitada. Ejemplo L=3

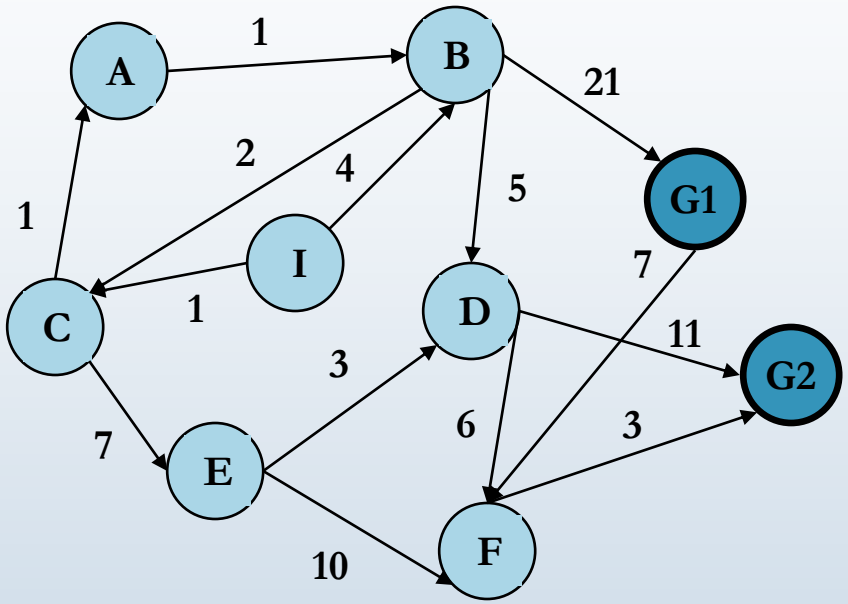


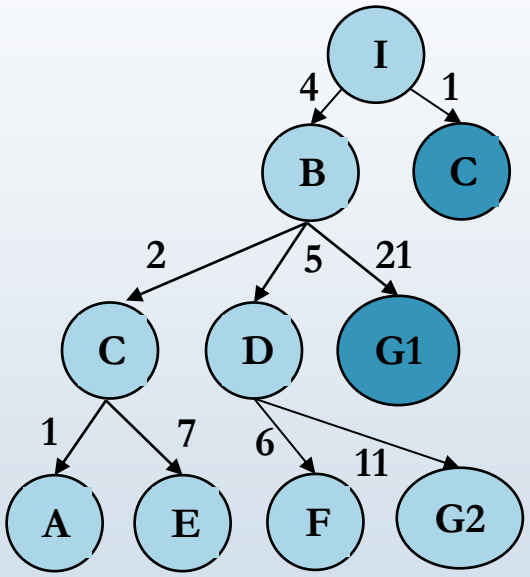


Pila de nodos abiertos: G2 D G1 C

Nodos cerrados: I B C A E D F

Búsqueda de profundidad limitada. Ejemplo L=3

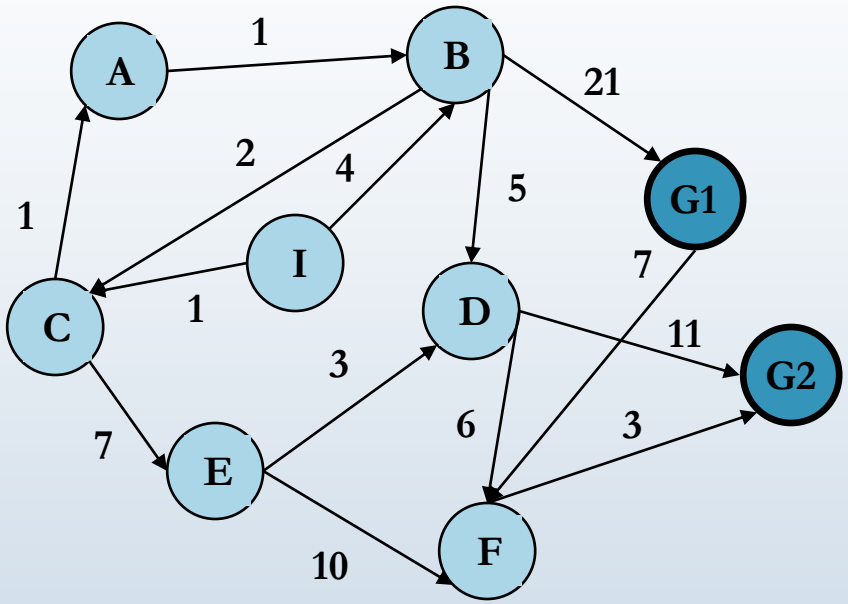


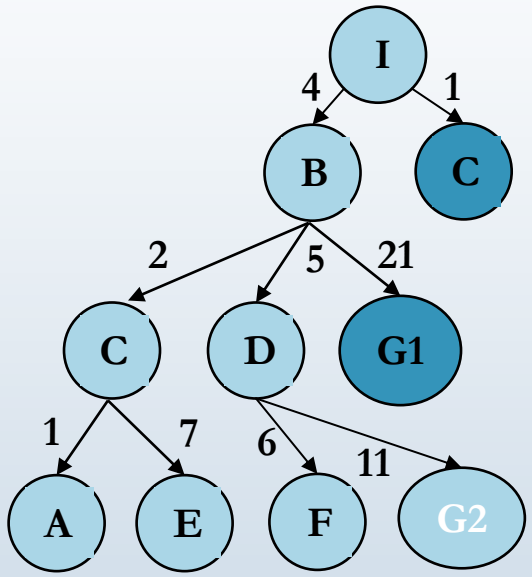


Pila de nodos abiertos: D G1 C

Nodos cerrados: I B C A E D F G2

Búsqueda de profundidad limitada. Ejemplo L=3





Pila de nodos abiertos: D G1 C

Nodos cerrados: I B C A E D F G2

Solución: I B D G2

Coste: 4+5+11 = 20

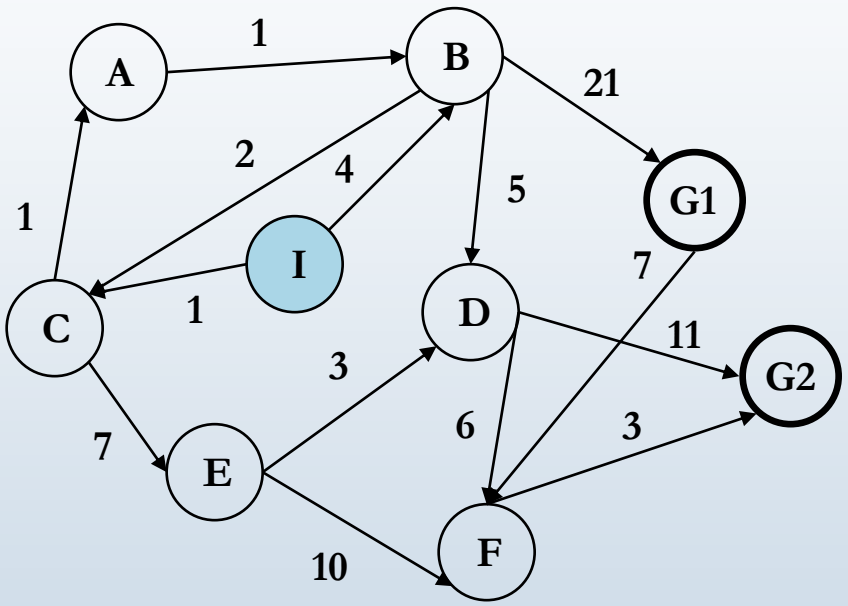
Búsqueda primero en profundidad con profundidad iterativa

- Aplicación iterativa del algoritmo de búsqueda de profundidad limitada, con límite de profundidad variando de forma creciente (0,1,...) → Evita el problema de la elección del límite.
- Combina las ventajas de los algoritmos de búsqueda primero en profundidad y primero en anchura.
- Suele ser el método no informado preferido cuando el espacio de estados es grande y la profundidad de la solución se desconoce.

Búsqueda primero en profundidad con profundidad iterativa. Evaluación

- Completo: SÍ, si existe solución siempre se encuentra.
- Óptimo: SÍ, la solución es óptima igual que en la búsqueda primero en anchura.
- Tiempo: $O(r^p)$ (mejor que primero en anchura)
  - Nodos generados: $p*r + (p-1) * r^2 + \ldots + (1)*r^p$
- Espacio: $O(r*p)$ (como primero en profundidad)
- El tratar repetidos acaba con todas las ventajas espaciales del algoritmo
  - Como máximo se puede utilizar la estructura de nodos abiertos para detectar bucles en el camino actual.

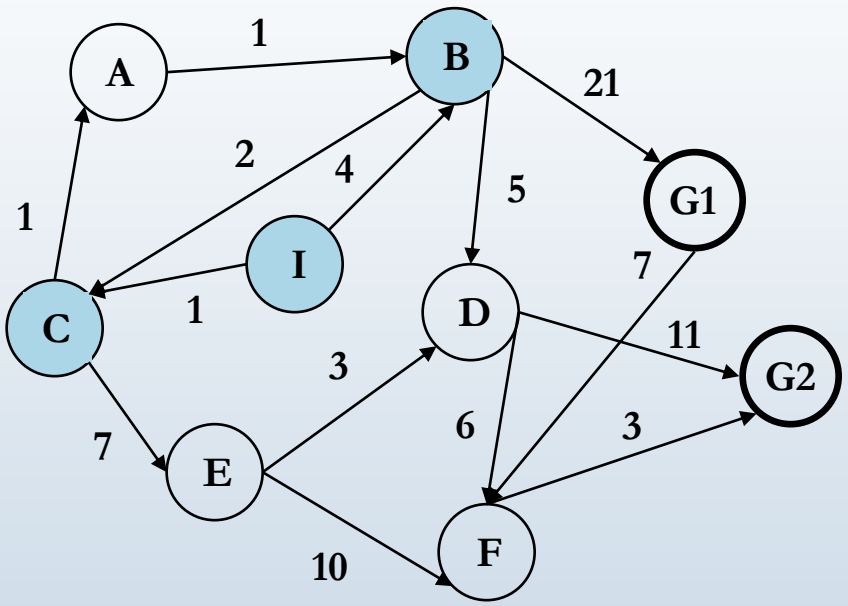
Búsqueda primero en profundidad con profundidad iterativa.



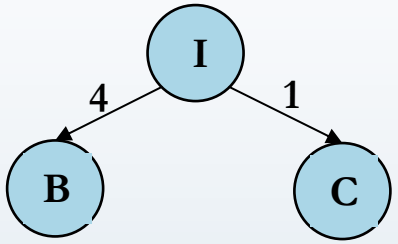
L = 0

Nodos expandidos: I

Búsqueda primero en profundidad con profundidad iterativa.

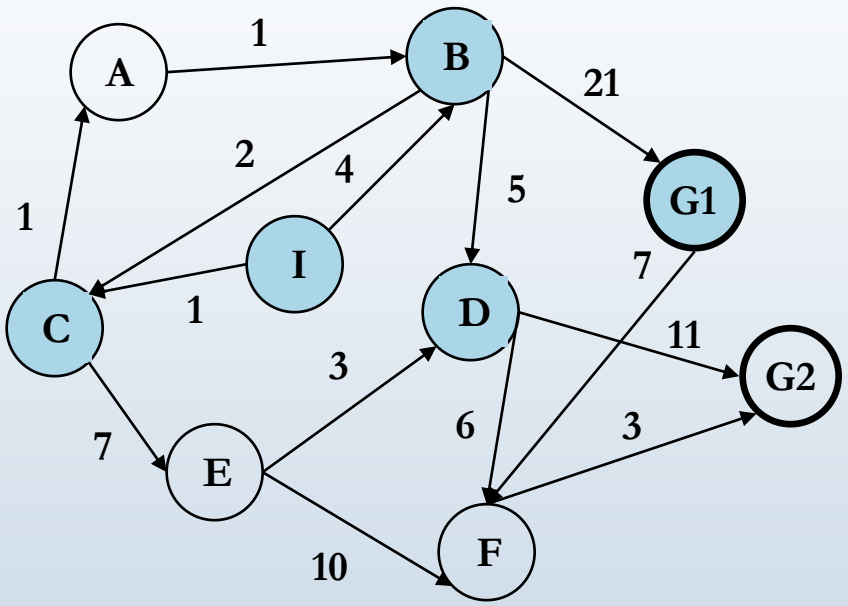


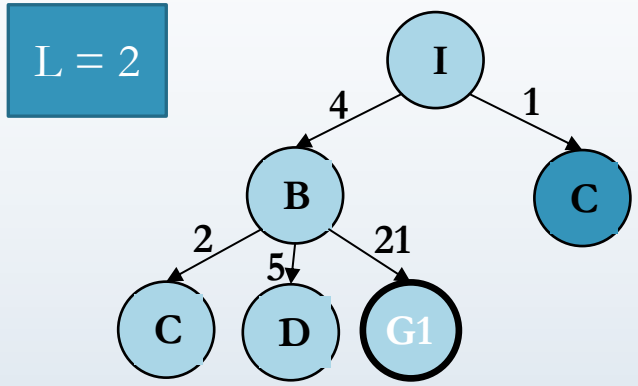
L = 1



Nodos expandidos: I I B C

Búsqueda primero en profundidad con profundidad iterativa.





Nodos expandidos: I I B C I B C D G1

Solución: I B G1

Coste: 4+21 = 25

Búsqueda bidireccional

- Ejecuta dos búsquedas simultáneas: una hacia delante desde el estado inicial y la otra hacia atrás desde el estado objetivo, parando cuando las dos búsquedas se encuentren en un mismo estado.
- Motivación: $r^{p/2} + r^{p/2}$ es mucho menor que r^p
- Es necesario
  - Conocer el estado objetivo (si hay varios, puede ser problemático).
  - Poder obtener los predecesores de un estado (operadores reversibles).

Búsqueda bidireccional

- No está fijado qué algoritmos de búsqueda habría que utilizar en ambos lados
  - Lo más lógico es que en uno de ellos se trabaje con búsqueda en anchura.
  - En el otro mejor un algoritmo con profundización iterativa para que no haya que mantener en memoria todos los nodos.

Búsqueda bidireccional. Evaluación

- Óptima y completa si las búsquedas son en anchura y los costes son iguales en todos los operadores.
  - Otras combinaciones no lo garantizan.
- Tiempo: $O(2*r^{p/2}) = O(r^{p/2})$
- Espacio: $O(r^{p/2})$
  - Al menos, los nodos de una de las dos partes deben mantenerse en memoria para la comparación.
    - Tabla hash para guardarlos.
  - Es la mayor debilidad del algoritmo.

Búsqueda no informada. Resumen

	BPA	BCU	BPP	BPL	BPI	BD
Completa	SI[1]	SI[1,3]	NO	SI ($L \geq p$)	SI[1]	SI[1,4]
Óptima	SI[2]	SI	NO	NO	SI[2]	SI[2,4]
Tiempo	$O(r^p)$	$O(r^{C^*/\varepsilon})$	$O(r^m)$	$O(r^L)$	$O(r^p)$	$O(r^{p/2})$
Espacio	$O(r^p)$	$O(r^{C^*/\varepsilon})$	$O(r*m)$	$O(r*L)$	$O(r*p)$	$O(r^{p/2})$

r : factor de ramificación; m : máxima profundidad del árbol;
p : profundidad mínima de la solución; L : límite de profundidad;
C* : coste de la solución óptima

1 : r es finito
2: coste de los operadores iguales, coste ≈ profundidad
3: costes ≥ ε, con ε positivo
4: ambas direcciones se utiliza BPA

Búsqueda no informada. Conclusiones

- Tienen un coste temporal que es una función exponencial del tamaño de la entrada.
  - El tiempo para encontrar la mejor solución a un problema no es asumible en problemas reales.
- Muy ineficientes en la mayoría de los casos.
- Ante la explosión combinatoria, la fuerza bruta es impracticable.

Bibliografía

- Russell, S.J., Norvig, P. 2021. Artificial Intelligence: A Modern Approach, 4th Global edition. Ed. Pearson Prentice Hall.
- Nilsson, N.J. 2001. Inteligencia Artificial. Una nueva síntesis. Ed. McGraw-Hill.